CodeCoach: A Socratic Hint Agent for First-Year Programming Labs

ELEC5623 Group Business Proposal · Track A — AI Agent Product

Group 12

Cover information	
Track	A — AI Agent Product
Group	12
Members	Daniel Ho (530201447), Grace Kim (530188320), Arjun Mehta (530176954), Chloe Martin (530203381), Felix Wong (530159902)
Lab tutor and approval	Dr K. Morrison — approved 27 August 2026: "Approved; keep to one introductory unit."
Product vision	Every first-year student stuck on a lab exercise gets a hint that moves them forward without handing them the answer.

1. Executive Summary

Introductory programming units enrol several hundred students per semester, and lab sessions are where most practical learning happens. In a two-hour lab, one tutor typically supports 25–30 students, and much of that time is spent on the same recurring difficulties: misunderstood error messages, off-by-one loops, and confusion between returning and printing a value. Students who wait too long for help disengage, and the evidence that early struggle predicts later failure in introductory programming is well established (Denny et al., 2024).

General-purpose AI assistants are now widely available to students, but they tend to produce complete solutions on request. Educators have raised concerns that this short-circuits learning, particularly for novices who cannot yet evaluate the code they receive (Kasneci et al., 2023; Prather et al., 2023).

This proposal presents **CodeCoach**, an AI agent that sits beside the lab exercise and responds to a stuck student with a graded sequence of hints. CodeCoach reads the exercise specification, the student's current code and the latest error or test output, diagnoses the most likely misconception, and responds with a question or hint at the lowest level of help that is likely to unblock the student. It escalates only when the student remains stuck, and it never outputs a complete solution. Tutors see a live summary of which misconceptions are occurring in the room.

The expected value is faster unblocking during labs, reduced repetitive load on tutors and a learning interaction that preserves productive struggle.

2. Introduction and Background

2.1 Application Context

Our context is a first-year Python unit with weekly two-hour labs. Each lab contains four to six short exercises with automated tests. Students work in a browser-based editor provided by the unit, and tutors circulate to help.

2.2 Evidence that the Problem Exists

Novice programmers spend a large share of their time interpreting error messages, and unhelpful error messages are a well-documented barrier in introductory programming (Becker et al., 2019). Studies of help-seeking in labs show that students often wait several minutes for tutor assistance, and that many do not ask at all (Denny et al., 2024). Tutors in our own unit, consulted informally, reported that roughly half of help requests concern the same handful of issues each week.

2.3 Current Practice and Existing Tools

Students currently rely on tutors, discussion forums and, increasingly, general AI chat assistants. Automated test feedback tells a student *that* a test failed but not *why*. Enhanced compiler error messages help with syntax but not with logic errors (Becker et al., 2019).

2.4 Learning Design Principles

Three principles from computing education research shape the design. First, **productive struggle**: novices learn more when they work through a difficulty with appropriate support than when they are shown the answer, provided the struggle does not tip into disengagement. Second, **graded assistance**: effective tutors start with the least help that might work — a question that redirects attention — and add detail only when needed. Third, **error-message literacy**: many novice difficulties are not conceptual but interpretive; the student does not know what the error message is telling them (Becker et al., 2019). Enhanced explanations of error messages generated by language models have been shown to be understandable to novices in many cases, although quality varies (Leinonen et al., 2023).

These principles imply that the system should (a) diagnose before it helps, (b) control the level of help explicitly, and (c) treat the student's own code and error output as the primary context rather than a generic description of the topic.

2.5 What Tutors Do Today

When a tutor reaches a stuck student, a typical interaction has four moves: the tutor asks what the student expected to happen, asks them to point to where the behaviour diverges, names the concept involved, and only then suggests a change. Tutors we consulted described the first two moves as the most valuable and the most time-consuming, because they require reading the student's code from scratch each time. CodeCoach is designed to perform these first moves consistently, so that tutor time is reserved for students who need a conversation rather than a pointer.

3. Problem Statement and Motivation

3.1 Specific Problem

During labs, students who are stuck on a logic or runtime error wait too long for help, and when they turn to general AI tools they often receive complete solutions that remove the learning opportunity.

3.2 Who Is Affected

First-year students, especially those with no prior programming experience; lab tutors, who handle repetitive questions; and unit coordinators, who need insight into common difficulties.

3.3 Why the Problem Matters

Early disengagement in introductory programming is associated with failure and withdrawal. Unblocking students quickly, without doing the work for them, is the central pedagogical challenge of lab support.

3.4 Why Generative AI Is Appropriate

Diagnosing a novice's misconception from their code, the exercise specification and an error trace requires flexible reasoning over code and natural language. Rule-based hint systems require hand-authored rules for every exercise and fail on unanticipated mistakes. Large language models can generate explanations of code and errors that novices find helpful (Leinonen et al., 2023), but unconstrained they also give away answers. A constrained agent that controls the level of help is therefore both necessary and distinct from a chat assistant.

3.5 Objective

Build a working hint agent integrated with one week's lab exercises, with graded hint levels, solution-leak prevention and a tutor dashboard.

4. Stakeholders and Requirements Engineering

4.1 Stakeholders

Stakeholder	Need
Student	Get unstuck quickly while still solving the problem themselves
Tutor	Spend less time on repetitive issues; see who needs help
Unit coordinator	Understand common misconceptions across labs
University	Responsible use of AI in assessment-adjacent activities

4.2 User Stories

- As a student, I want a hint about why my loop fails, so that I can fix it myself.
- As a student, I want more help if the first hint is not enough.
- As a tutor, I want to see which students have been stuck the longest.
- As a coordinator, I want a summary of misconceptions per exercise.

4.3 Scope

In scope: Python exercises in one introductory unit; hints for runtime, logic and test failures; a tutor dashboard. Out of scope: assessment tasks, plagiarism detection, languages other than Python.

4.4 Functional Requirements

ID	Requirement	Priority	Owner	Acceptance criterion
FR1	Read the exercise specification and tests	Must	A1	Specification and test names available to the agent for all test exercises
FR2	Read the student's current code and last run output	Must	A1	Latest code and output captured on hint request
FR3	Store exercise metadata in a local catalogue	Should	A1	
FR4	Diagnose the most likely misconception	Must	A2	Diagnosis matches tutor label in most cases
FR5	Generate a level-1 hint (question)	Must	A2	Hint contains no code from the reference solution
FR6	Escalate to level-2 and level-3 hints on repeated requests	Must	A2	Level increases only after a further request
FR7	Never output a complete solution	Must	A3	Leak checker blocks outputs with >60% token overlap with the reference solution

ID	Requirement	Priority	Owner	Acceptance criterion
FR8	Check hints against the reference solution before display	Must	A3	All hints pass the leak checker
FR9	Log every hint with exercise, level and diagnosis	Must	A3	Log entry for every hint
FR10	Show a tutor dashboard of students waiting	Must	A4	Dashboard updates within 10 seconds
FR11	Show misconception counts per exercise	Should	A4	
FR12	Let the tutor mark a student as helped	Could	A4	
FR13	Provide a hint button inside the lab editor	Must	A5	Button visible on every exercise
FR14	Let the student rate a hint as helpful or not	Should	A5	Rating stored with the hint
FR15	Let the student request a tutor instead	Should	A5	Request appears on the tutor dashboard

4.5 Non-Functional Requirements

ID	Requirement
NFR1	Hints should appear quickly (target under 8 seconds)
NFR2	The interface should be easy for first-year students to use
NFR3	Student code is not used to train external models

4.6 Constraints

The prototype uses one week's lab exercises and a hosted model through the university's approved endpoint.

5. Proposed Product and Innovation

5.1 Product Concept

CodeCoach adds a "Hint" button to the lab editor. When pressed, the agent gathers context, diagnoses the problem and returns a hint at the current level. Each further press for the same exercise raises the level: level 1 asks a guiding question, level 2 points to the relevant line or concept, level 3 explains the fix in words but still without writing the code.

5.2 AI Components

1. **Context builder** assembles the specification, tests, student code and last output.
2. **Diagnosis step** asks the model to name the most likely misconception from a fixed taxonomy of 18 common novice errors, adapted from the literature on novice misconceptions.
3. **Hint generator** produces a hint at the requested level, conditioned on the diagnosis.
4. **Leak checker** compares the hint with the reference solution (token overlap and code-block detection) and regenerates once if the check fails; if it fails again, a fixed level-appropriate message is shown.

5.2a Misconception Taxonomy

The diagnosis step selects from a fixed taxonomy so that hints and tutor analytics use consistent categories. The initial taxonomy is:

Code	Misconception	Typical symptom
M1	Printing instead of returning	Test receives `None`
M2	Off-by-one loop bounds	Last element missing or index error
M3	Modifying a list while iterating	Elements skipped
M4	Integer vs float division	Unexpected truncation
M5	Variable scope confusion	`NameError` or unchanged outer variable
M6	Shadowing a built-in name	`TypeError: 'list' object is not callable`
M7	Mutable default argument	State persists across calls
M8	String vs number comparison	Comparison always false
M9	Missing base case in recursion	`RecursionError`
M10	Assignment vs equality	Syntax error or wrong branch
M11	Early return inside loop	Only first item processed
M12	Misreading the specification	Output format differs from expected
M13	Indentation changes control flow	Statement runs once instead of per iteration
M14	Uninitialised accumulator	Wrong totals
M15	Confusing index and value	Wrong element used
M16	Incorrect boolean logic	Condition inverted or too broad
M17	Input parsing errors	`ValueError` on conversion
M18	Other / unclear	Escalate to tutor suggestion

5.2b Hint Levels

Level	Purpose	Form	Example constraint
1	Redirect attention	A question about expected vs actual behaviour	No line numbers, no code
2	Locate	Points to the relevant line or concept	May name the line; no corrected code
3	Explain the fix in words	Describes what to change and why	Never writes the corrected statement

5.3 Distinction from a Generic Chatbot

A chat assistant answers whatever is asked, usually with full code. CodeCoach controls the level of help, grounds the hint in the exercise's own tests, checks every hint against the reference solution before display, and aggregates misconceptions for tutors.

6. Methodology and System Design

6.1 Architecture

The lab editor (existing unit platform) calls a small FastAPI service. The service holds the exercise catalogue, calls the model for diagnosis and hint generation, runs the leak checker, and writes to a PostgreSQL log that also feeds the tutor dashboard (Appendix A).

6.2 Model Strategy

We will use a hosted model at low temperature for diagnosis (structured output over the taxonomy) and slightly higher temperature for hint wording. Prompts include two worked examples per hint level.

6.3 Development Method

Scrum with two-week sprints, a shared backlog and weekly stand-ups. Each sprint ends with a demo to the group.

6.4 Data and Tools

Exercises and reference solutions come from the unit's public lab sheets. Student code for development will be written by our group to imitate typical mistakes. Tools: Python, FastAPI, PostgreSQL, React, a hosted LLM API.

6.5 Prompt Design and Context Limits

The diagnosis prompt contains the exercise specification, the names and messages of failing tests, the student's code with line numbers and the taxonomy with one-line definitions. It asks for a JSON object with the misconception code, the line numbers involved and a one-sentence justification. The hint prompt receives the diagnosis, the hint level and the level's constraints, plus two worked examples at that level. Student code longer than 200 lines is truncated around the lines referenced by the failing test's traceback; in our exercises no solution exceeds 60 lines, so truncation is expected to be rare.

6.6 Leak Checking in Detail

The leak checker has two parts. A structural check rejects any hint containing a fenced code block or more than one line that parses as Python. A similarity check tokenises the hint and the reference solution and rejects hints whose overlap with the solution's identifier and operator sequence exceeds the threshold. Both checks are deterministic and logged, so that every blocked hint can be inspected.

6.7 Tutor Dashboard

The dashboard lists students in the lab session ordered by time since their first hint request on the current exercise, with the current hint level and the diagnosed misconception. A summary panel shows misconception counts per exercise for the session, which tutors can use to decide when a short whole-class explanation would be more efficient than individual help.

7. Business and Product Analysis

7.1 Value Proposition

CodeCoach helps students get unstuck and helps tutors focus their time.

7.2 Users and Customers

Students and tutors in introductory programming units; the customer would be the faculty's teaching support team.

7.3 Alternatives

We are not aware of another tool that offers graded hints for lab exercises, so CodeCoach fills a clear gap.

7.4 Feasibility and Costs

The service is small and the model calls are short, so cost per hint will be low. The unit platform already supports embedding a button in the editor.

8. Evaluation Plan

We will evaluate CodeCoach by demonstrating it to our classmates and tutor in Week 12 and collecting their feedback. After using the hint button on a few exercises, participants will complete a short survey

asking whether the hints were helpful and whether they would use CodeCoach in their labs.

The project will be considered successful if most participants find the hints helpful and the demo runs without errors. We will also look through the hint logs to check that hints look reasonable.

9. Risks and Responsible AI

Risk	Note
Hints reveal the solution	Leak checker
Wrong diagnosis	Student can ask for another hint
Model unavailable	Show a message
Privacy of student code	Use approved endpoint
Students over-rely on hints	Limit hint levels

10. Semester Plan and Team Responsibilities

Week	Milestone
5–6	Exercise catalogue, context builder
7–8	Diagnosis and hint generation
9	Leak checker
10	Tutor dashboard
11	Integration with the lab editor, misconception analytics, student ratings
12	Demo and survey
13	Final presentation

Area	Member	Responsibilities
A1	Daniel Ho	Exercise catalogue and context builder
A2	Grace Kim	Diagnosis and hint generation
A3	Arjun Mehta	Leak checker and logging
A4	Chloe Martin	Tutor dashboard
A5	Felix Wong	Editor integration and student feedback

11. References

Becker, B. A., Denny, P., Pettit, R., Bouchard, D., Bouvier, D. J., Harrington, B., Kamil, A., Karkare, A., McDonald, C., Osera, P.-M., Pearce, J. L., & Prather, J. (2019). Compiler error messages considered unhelpful: The landscape of text-based programming error message research. In *Proceedings of the Working Group Reports on Innovation and Technology in Computer Science Education* (pp. 177–210). ACM.

Denny, P., Prather, J., Becker, B. A., Finnie-Ansley, J., Hellas, A., Leinonen, J., Luxton-Reilly, A., Reeves, B. N., Santos, E. A., & Sarsa, S. (2024). Computing education in the era of generative AI. *Communications of the ACM, 67*(2), 56–67.

Kasneci, E., Sessler, K., Küchemann, S., Bannert, M., Dementieva, D., Fischer, F., Gasser, U., Groh, G., Günnemann, S., Hüllermeier, E., Krusche, S., Kutyniok, G., Michaeli, T., Nerdel, C., Pfeffer, J., Poquet,

O., Sailer, M., Schmidt, A., Seidel, T., … Kasneci, G. (2023). ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and Individual Differences, 103*, 102274.

Leinonen, J., Hellas, A., Sarsa, S., Reeves, B., Denny, P., Prather, J., & Becker, B. A. (2023). Using large language models to enhance programming error messages. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education* (pp. 563–569). ACM.

Prather, J., Denny, P., Leinonen, J., Becker, B. A., Albluwi, I., Craig, M., Keuning, H., Kiesler, N., Kohn, T., Luxton-Reilly, A., MacNeil, S., Petersen, A., Pettit, R., Reeves, B. N., & Savelka, J. (2023). The robots are here: Navigating the generative AI revolution in computing education. In *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education* (pp. 108–159). ACM.

Appendix A — Architecture

Component	Responsibility
Lab editor (existing)	Hint button; sends code and output
Hint service (FastAPI)	Context building, model calls, leak check
Model endpoint	Diagnosis and hint generation
PostgreSQL	Hint log, ratings, tutor dashboard data
Tutor dashboard (React)	Waiting students, misconception counts

Appendix B — Example Interactions

A student's function returns `None` because it prints the result instead of returning it. Level 1: "Your test expects a value back from `average()`. What does your function give back to the caller right now?" Level 2: "Look at the last line of `average()`: it displays the result. Is displaying the same as giving it back?" Level 3: "Replace the line that displays the result with one that returns it, then run the tests again."

A second student's loop `for i in range(1, len(scores))` misses the first score. Level 1: "Which positions in the list does your loop visit? Try printing `i` on each pass." Level 2: "The loop starts at position 1. Where does a Python list start?" Level 3: "Change the start of the range so the first position is included."

Appendix C — Diagnosis Output Contract

The diagnosis step must return `{"code": "M1".."M18", "lines": [int], "justification": str}`. Outputs that do not parse, or that name a code outside the taxonomy, are retried once and otherwise mapped to M18, which produces a level-1 hint suggesting the student ask their tutor.